

lua-xlsx - Documentation française

Julien Freyermuth

Table des matières

1	lua-xlsx - Documentation française	2
1.1	Table des matières	2
1.2	Architecture et intégration Babet	2
1.3	Conventions	2
1.4	Écriture XLSX	3
1.4.1	xlsx.new([opts]) -> workbook	3
1.4.2	workbook:add_sheet([name]) -> sheet	3
1.4.3	sheet:write(row, col, value [, style]) -> sheet	3
1.4.4	sheet:append_row(values [, style]) -> sheet	4
1.4.5	sheet:write_rows(matrix [, style]) -> sheet	4
1.5	Styles, structure, mise en page, hyperliens et formules	4
1.5.1	xlsx.VERSION	4
1.5.2	xlsx.style([opts]) -> style	4
1.5.3	xlsx.is_style(value) et xlsx.style_options(style)	6
1.5.4	sheet:set_style(row, col, style) -> sheet	6
1.5.5	sheet:set_column_width(col, width) -> sheet	6
1.5.6	sheet:set_row_height(row, height) -> sheet	6
1.5.7	sheet:freeze_panes(rows, cols) -> sheet	6
1.5.8	sheet:set_auto_filter([range]) -> sheet	7
1.5.9	Cellules fusionnées	7
1.5.10	Hyperliens	7
1.5.11	Formules	8
1.5.12	Exemple combiné 1.2.0	8
1.5.13	workbook:save(path [, opts]) -> true, nil nil, err	9
1.5.14	xlsx.write_rows(path, matrix [, opts])	10
1.6	Dates 1900 et 1904	10
1.6.1	xlsx.date(year, month, day)	10
1.6.2	xlsx.datetime(year, month, day [, hour, minute, second])	10
1.6.3	Utilitaires	10
1.7	Lecture XLSX	10
1.7.1	xlsx.open(path [, opts]) -> workbook, nil nil, err	10
1.7.2	workbook:date_system() -> "1900" "1904"	11
1.7.3	workbook:sheet_names() -> { string, ... }	11
1.7.4	workbook:sheet(which) -> sheet nil [, err]	11
1.7.5	Valeurs et formules	11
1.7.6	Lecture des styles	11
1.7.7	Lecture de la mise en page	12
1.7.8	Lecture des cellules fusionnées	12
1.7.9	Lecture des hyperliens	12
1.7.10	Dimensions et itération	13
1.8	Limites et validation ZIP	13
1.9	DataFrame	14
1.9.1	Construction	14
1.9.2	Introspection	14
1.9.3	Transformations non destructives	14
1.9.4	Groupement et agrégations	15
1.9.5	Sorties	15
1.10	Contrat d'erreur	16

1.11 Tests et interopérabilité	16
1.12 Génération des PDF	17
1.13 Limites connues	17

1 lua-xlsx - Documentation française

Référence des modules `xlsx` et `dataframe`. La cible principale est Babet avec Lua 5.5, mais le cœur reste compatible avec Lua standard 5.3+.

Documentation de lua-xlsx 1.2.0.

1.1 Table des matières

- [Architecture et intégration Babet](#)
- [Conventions](#)
- [Écriture XLSX](#)
- [Styles, structure, mise en page, hyperliens et formules](#)
- [Dates 1900 et 1904](#)
- [Lecture XLSX](#)
- [Limites et validation ZIP](#)
- [DataFrame](#)
- [Contrat d'erreur](#)
- [Tests et interopérabilité](#)
- [Génération des PDF](#)
- [Limites connues](#)

1.2 Architecture et intégration Babet

`xlsx.lua` et `dataframe.lua` n'exigent aucun module externe. Lorsque la globale `babet` existe, `xlsx.lua` utilise automatiquement les fonctions suivantes :

Fonction Babet	Usage dans lua-xlsx
<code>babet.writeFileAtomic</code>	publication atomique et durable d'un XLSX
<code>babet.archive.test</code>	validation complète du ZIP avant lecture
<code>babet.fileSize</code>	contrôle de taille avant chargement en mémoire
<code>babet.crc32</code>	calcul natif du CRC-32 lors de l'écriture et de la lecture

L'absence d'une fonction précise ne bloque pas la bibliothèque. Le chemin Lua pur reste disponible pour cette opération.

Pour désactiver volontairement l'intégration Babet lors d'un appel :

```
assert(wb:save("rapport.xlsx", { use_babet = false }))

local read, err = xlsx.open("rapport.xlsx", {
    use_babet = false,
})
assert(read, err)
```

Cette option est utile pour tester le chemin de repli. En production sous Babet, il est généralement préférable de conserver la valeur par défaut `true`.

1.3 Conventions

- `sheet:write(row, col, value [, style])` et `sheet:read(row, col)` utilisent des indices **0-based**. A1 correspond à (0, 0).

- `sheet:rows()` produit des lignes Lua **1-based**. `row[1]` correspond à la colonne A et `row.n` indique la largeur.
- Une cellule vide vaut `nil`. Le booléen `false` reste distinct de `nil`.
- Les dates lues sont des chaînes ISO 8601.
- Les lignes et colonnes respectent les limites XLSX : lignes 0..1048575 et colonnes 0..16383.
- Les chaînes écrites doivent être en UTF-8 valide et ne contenir que des caractères permis par XML 1.0.
- Les transformations DataFrame renvoient de nouveaux records ; elles ne partagent pas les tables de lignes du DataFrame source.

1.4 Écriture XLSX

1.4.1 `xlsx.new([opts]) -> workbook`

Crée un classeur vide.

Option :

Option	Défaut	Valeurs
<code>date_system</code>	"1900"	"1900" ou "1904"

Exemple 1900 :

```
local wb = xlsx.new()
```

Exemple 1904 :

```
local wb = xlsx.new({ date_system = "1904" })
```

Une option inconnue est refusée.

1.4.2 `workbook:add_sheet([name]) -> sheet`

Ajoute une feuille. Le nom par défaut est SheetN.

Règles :

- de 1 à 31 **caractères Unicode**, et non 31 octets ;
- UTF-8 et XML 1.0 valides ;
- pas de `:`, `\`, `/`, `?`, `*`, `[` ou `]` ;
- pas d'apostrophe au début ou à la fin ;
- pas de doublon exact ou de doublon ASCII ne différant que par la casse.

```
local wb = xlsx.new()
local fr = wb:add_sheet("Données")
local unicode = wb:add_sheet(string.rep("é", 20))
```

1.4.3 `sheet:write(row, col, value [, style]) -> sheet`

Types acceptés :

- `string` ;
- entier ou flottant fini ;
- `boolean` ;
- valeur créée par `xlsx.date` ou `xlsx.datetime` ;
- formule créée par `xlsx.formula` ;
- `nil` pour une cellule vide, éventuellement stylée.

```
local sh = xlsx.new():add_sheet("Exemple")
sh:write(0, 0, "Texte")
sh:write(0, 1, 42)
sh:write(0, 2, 3.14)
```

```
sh:write(0, 3, false)
sh:write(0, 4, xlsx.date(2026, 7, 18))
sh:write(0, 5, xlsx.formula("SUM(B1:C1)"))
```

NaN, les infinis, les types non pris en charge, les indices négatifs et les indices hors limites Excel lèvent une erreur Lua.

1.4.4 sheet:append_row(values [, style]) -> sheet

Ajoute une ligne après la dernière ligne connue.

```
sh:append_row({ "Alice", 30, true })
sh:append_row({ "Bob", 25, false })
```

Les nil terminaux ne peuvent pas être distingués de l'absence d'élément dans un tableau Lua. Pour écrire une cellule éloignée, utiliser write().

1.4.5 sheet:write_rows(matrix [, style]) -> sheet

Ajoute une matrice complète :

```
sh:write_rows({
  { "Nom", "Score" },
  { "Alice", 18 },
  { "Bob", 15 },
})
```

1.5 Styles, structure, mise en page, hyperliens et formules

La version 1.2.0 écrit ces informations et expose également leur sous-ensemble pris en charge dans l'API de lecture. Les objets style, formula et hyperlink sont immuables.

1.5.1 xlsx.VERSION

```
assert(xlsx.VERSION == "1.2.0")
```

1.5.2 xlsx.style([opts]) -> style

Un style doit être créé avec xlsx.style; une table brute n'est pas acceptée par write() ou set_style().

Option	Type	Valeurs
bold	booléen	police grasse
italic	booléen	police italique
underline	chaîne	none, single, double
strike	booléen	texte barré
font_name	chaîne	nom UTF-8 non vide, 255 octets maximum
font_size	nombre	taille de 1 à 409 points
font_color	chaîne	RGB RRGGBB, #RRGGBB ou ARGB AARRGGBB
fill_color	chaîne	fond plein RGB ou ARGB
horizontal	chaîne	left, center, right, justify
vertical	chaîne	top, center, bottom, justify
wrap_text	booléen	retour automatique à la ligne
number_format	chaîne	alias ou code de format Excel
border	table	côtés left, right, top, bottom

Les couleurs RGB à six chiffres reçoivent automatiquement le canal alpha opaque FF.

Gras et italique :

```
local emphasis = xlsx.style({ bold = true, italic = true })
sh:write(0, 0, "Important", emphasis)
```

Nom et taille de police :

```
local title = xlsx.style({
  font_name = "Liberation Sans",
  font_size = 16,
})
sh:write(1, 0, "Titre", title)
```

Soulignement simple ou double :

```
sh:write(2, 0, "Simple", xlsx.style({ underline = "single" }))
sh:write(3, 0, "Double", xlsx.style({ underline = "double" }))
```

Texte barré :

```
sh:write(4, 0, "Ancienne valeur", xlsx.style({ strike = true }))
```

Couleurs de police et de fond :

```
local colored = xlsx.style({
  font_color = "FFFFFF",
  fill_color = "4472C4",
})
sh:write(5, 0, "Couleurs", colored)
```

Alignement et retour à la ligne :

```
local wrapped = xlsx.style({
  horizontal = "center",
  vertical = "center",
  wrap_text = true,
})
sh:write(6, 0, "Texte long", wrapped)
```

Formats numériques prédéfinis :

Alias	Code Excel produit
general	aucun format personnalisé
integer	0
decimal	0.00
percent	0.00%
currency_eur	#,##0.00 "€"
currency_usd	\$#,##0.00
date	yyyy-mm-dd
datetime	yyyy-mm-dd hh:mm:ss

```
sh:write(0, 1, 42, xlsx.style({ number_format = "integer" }))
sh:write(1, 1, 3.14159, xlsx.style({ number_format = "decimal" }))
sh:write(2, 1, 0.125, xlsx.style({ number_format = "percent" }))
sh:write(3, 1, 19.90, xlsx.style({ number_format = "currency_eur" }))
```

Un code personnalisé est accepté :

```
sh:write(4, 1, 12.3456, xlsx.style({
  number_format = '0.000 "kg"',
}))
```

Une date reçoit automatiquement son format si le style n'en définit pas :

```
sh:write(0, 2, xlsx.date(2026, 7, 18), xlsx.style({ bold = true }))
```

1.5.2.1 Bordures Chaque côté est une table contenant style et, facultativement, color. Styles pris en charge : thin, medium, thick, dashed, dotted et double.

Bordure fine inférieure :

```
local bottom = xlsx.style({
  border = { bottom = { style = "thin" } },
})
```

Bordure colorée :

```
local red_left = xlsx.style({
  border = { left = { style = "medium", color = "FF0000" } },
})
```

Quatre côtés combinés :

```
local framed = xlsx.style({
  border = {
    left = { style = "thin", color = "808080" },
    right = { style = "thin", color = "808080" },
    top = { style = "double", color = "000000" },
    bottom = { style = "double", color = "000000" },
  },
})
```

1.5.3 xlsx.is_style(value) et xlsx.style_options(style)

```
assert(xlsx.is_style(framed))
local opts = xlsx.style_options(framed)
print(opts.border.top.style)
```

style_options() renvoie une copie profonde. Modifier cette table ne modifie pas le style d'origine.

1.5.4 sheet:set_style(row, col, style) -> sheet

```
sh:write(0, 0, "Titre")
sh:set_style(0, 0, xlsx.style({ bold = true }))
sh:set_style(1, 0, xlsx.style({ fill_color = "E2F0D9" })) -- cellule vide
sh:set_style(0, 0, nil) -- retire le style
```

1.5.5 sheet:set_column_width(col, width) -> sheet

col est 0-based et width doit être compris entre 0.1 et 255.

```
sh:set_column_width(0, 24)
sh:set_column_width(1, 14)
```

1.5.6 sheet:set_row_height(row, height) -> sheet

La hauteur en points doit être comprise entre 0.1 et 409.5.

```
sh:set_row_height(0, 28)
```

1.5.7 sheet:freeze_panes(rows, cols) -> sheet

```
sh:freeze_panes(1, 0) -- première ligne
sh:freeze_panes(0, 1) -- première colonne
sh:freeze_panes(1, 1) -- ligne et colonne, cellule libre B2
sh:freeze_panes(0, 0) -- supprime le gel
```

1.5.8 sheet:set_auto_filter([range]) -> sheet

```
sh:set_auto_filter()           -- zone utilisée à la sauvegarde
sh:set_auto_filter("A1:D100")
sh:set_auto_filter(false)      -- désactive
```

Le filtre ajoute les boutons Excel mais ne masque aucune ligne lui-même.

1.5.9 Cellules fusionnées

1.5.9.1 sheet:merge_cells(range) -> sheet

```
sh:write(0, 0, "Rapport")
sh:merge_cells("A1:D1")
```

1.5.9.2 sheet:merge_cells(row1, col1, row2, col2) -> sheet

 Coordonnées 0-based :

```
sh:merge_cells(2, 0, 2, 3) -- A3:D3
```

Les fusions d'une seule cellule, inversées ou chevauchantes sont refusées. Seule la cellule supérieure gauche conserve sa valeur, son style et son lien ; les autres cellules de la plage sont vidées.

1.5.9.3 sheet:unmerge_cells(range) -> sheet

```
sh:unmerge_cells("A1:D1")
```

Les anciennes valeurs supprimées par la fusion ne sont pas restaurées.

1.5.10 Hyperliens

1.5.10.1 xlsx.hyperlink(target [, text [, opts]]) -> hyperlink

 Lien externe directement écrit dans une cellule :

```
sh:write(0, 0, xlsx.hyperlink(
  "https://example.com/?a=1&b=2",
  "Ouvrir le site",
  { tooltip = "Site externe" }
))
```

Lien interne :

```
sh:write(1, 0, xlsx.hyperlink(
  "'Détails'!A1",
  "Voir les détails",
  { internal = true }
))
```

1.5.10.2 sheet:set_hyperlink(row, col, target [, opts]) -> sheet

 Applique un lien à une valeur déjà écrite :

```
sh:write(2, 0, "Documentation")
sh:set_hyperlink(2, 0, "https://example.com/docs")
```

Lien interne sur une cellule existante :

```
sh:set_hyperlink(3, 0, "'Résumé'!B2", { internal = true })
```

1.5.10.3 sheet:remove_hyperlink(row, col) -> sheet

```
sh:remove_hyperlink(2, 0)
```

La valeur de la cellule est conservée.

1.5.11 Formules

1.5.11.1 `xlsx.formula(expression [, cached_value])` -> **formula** Le signe = initial est facultatif :

```
sh:write(3, 1, xlsx.formula("SUM(B1:B3)"))
sh:write(3, 2, xlsx.formula("=AVERAGE(C1:C3)"))
```

Une valeur mise en cache facultative peut être une chaîne, un nombre ou un booléen :

```
sh:write(4, 1, xlsx.formula("SUM(B2:B4)", 42.5))
```

Cette valeur permet aux lecteurs `data_only` de voir un résultat avant un recalcul, mais `lua-xlsx` ne vérifie pas qu'elle correspond réellement à la formule. Sans cache, Excel ou LibreOffice calculera la formule à l'ouverture.

Une formule peut recevoir un style :

```
local money = xlsx.style({ number_format = "currency_eur" })
sh:write(10, 3, xlsx.formula("SUM(D2:D10)", 125.40), money)
```

`xlsx.is_formula(value)` reconnaît les objets de formule.

1.5.12 Exemple combiné 1.2.0

```
local xlsx = require("xlsx")
```

```
local wb = xlsx.new()
```

```
local sh = wb:add_sheet("Ventes")
```

```
local title = xlsx.style({
    bold = true,
    underline = "double",
    font_name = "Liberation Sans",
    font_size = 16,
    font_color = "FFFFFF",
    fill_color = "4472C4",
    horizontal = "center",
    vertical = "center",
    border = {
        bottom = { style = "double", color = "1F4E78" },
    },
})
```

```
local header = xlsx.style({
    bold = true,
    fill_color = "D9EAF7",
    horizontal = "center",
    border = {
        left = { style = "thin", color = "808080" },
        right = { style = "thin", color = "808080" },
        top = { style = "thin", color = "808080" },
        bottom = { style = "thin", color = "808080" },
    },
})
```

```
local money = xlsx.style({ number_format = "currency_eur" })
```

```
sh:write(0, 0, "Rapport des ventes", title)
sh:merge_cells("A1:E1")
sh:append_row({ "Produit", "Prix", "Quantité", "Total", "Lien" }, header)
sh:append_row({ "Clavier", 39.90, 2 })
sh:append_row({ "Souris", 24.50, 3 })
sh:write(2, 1, 39.90, money)
sh:write(3, 1, 24.50, money)
sh:write(2, 3, xlsx.formula("B3*C3", 79.80), money)
```

```

sh:write(3, 3, xlsx.formula("B4*C4", 73.50), money)
sh:write(2, 4, xlsx.hyperlink("https://example.com/clavier", "Fiche"))
sh:write(3, 4, "Résumé")
sh:set_hyperlink(3, 4, "'Résumé'!A1", { internal = true })

sh:set_column_width(0, 24)
sh:set_column_width(1, 14)
sh:set_column_width(2, 12)
sh:set_column_width(3, 16)
sh:set_column_width(4, 18)
sh:set_row_height(0, 30)
sh:freeze_panes(2, 1)
sh:set_auto_filter("A2:E4")

wb:add_sheet("Résumé"):write(0, 0, "Destination interne")
assert(wb:save("ventes.xlsx"))

```

1.5.13 workbook:save(path [, opts]) -> true, nil | nil, err

Options :

Option	Défaut	Effet
overwrite	true	remplace un fichier existant
durable	true	demande la synchronisation durable sous Babet
permissions	0644	permissions du fichier sous Babet
use_babet	true	utilise writeFileAtomic si disponible

Écriture simple :

```

local ok, err = wb:save("rapport.xlsx")
assert(ok, err)

```

Refuser un remplacement :

```

local ok, err = wb:save("rapport.xlsx", {
    overwrite = false,
})

```

Cache restructurable sans attente de durabilité :

```

assert(wb:save("cache.xlsx", {
    durable = false,
}))

```

Permissions privées :

```

assert(wb:save("privé.xlsx", {
    permissions = tonumber("600", 8),
}))

```

Exemple combiné :

```

local ok, err = wb:save("export/rapport.xlsx", {
    overwrite = true,
    durable = true,
    permissions = tonumber("640", 8),
    use_babet = true,
})
assert(ok, err)

```

Sous Babet, l'appel passe par `babet.writeFileAtomic`. Sous Lua standard, le module écrit un fichier temporaire dans le même dossier, contrôle `write`, `flush` et `close`, puis le renomme. Ce

repli fournit une publication atomique sur les systèmes POSIX usuels, mais ne reproduit pas toutes les garanties de confinement et de fsync de Babet.

1.5.14 `xlsx.write_rows(path, matrix [, opts])`

Options :

- `sheet` : nom de feuille, défaut `Sheet1` ;
- `date_system` : 1900 ou 1904 ;
- `overwrite`, `durable`, `permissions`, `use_babet` : mêmes options que `save()`.

```
assert(xlsx.write_rows("résultat.xlsx", {
  { "x", "y" },
  { 1, 2 },
  { 3, 4 },
}, {
  sheet = "Résultats",
  date_system = "1900",
  overwrite = true,
}))
```

1.6 Dates 1900 et 1904

1.6.1 `xlsx.date(year, month, day)`

```
sh:write(0, 0, xlsx.date(2026, 7, 18))
```

1.6.2 `xlsx.datetime(year, month, day [, hour, minute, second])`

```
sh:write(0, 1, xlsx.datetime(2026, 7, 18, 13, 45, 30))
```

Les constructeurs valident :

- année 1..9999 ;
- mois 1..12 ;
- nombre de jours réel du mois, années bissextiles comprises ;
- heure 0..23, minute et seconde 0..59 ;
- arguments entiers.

1.6.3 Utilitaires

```
local serial1900 = xlsx.date_to_serial(2026, 7, 18)
local serial1904 = xlsx.date_to_serial(2026, 7, 18, 0, 0, 0, true)

print(xlsx.serial_to_iso(serial1900, false))
print(xlsx.serial_to_iso(serial1904, false, true))
```

Le classeur écrit le bon numéro de série selon `date_system`. À la lecture, `<workbookPr date1904="1"/>` est détecté automatiquement.

1.7 Lecture XLSX

1.7.1 `xlsx.open(path [, opts]) -> workbook, nil | nil, err`

L'ouverture contrôle la taille, valide éventuellement l'archive avec Babet, vérifie le ZIP, les CRC et les XML, puis charge les métadonnées nécessaires.

```
local wb, err = xlsx.open("rapport.xlsx")
assert(wb, err)
```

Désactiver seulement la prévalidation Babet :

```
local wb, err = xlsx.open("rapport.xlsx", {
    validate_archive = false,
})
```

Le parseur Lua conserve ses propres vérifications.

1.7.2 workbook:date_system() -> "1900" | "1904"

```
print(wb:date_system())
```

1.7.3 workbook:sheet_names() -> { string, ... }

```
for _, name in ipairs(wb:sheet_names()) do print(name) end
```

1.7.4 workbook:sheet(which) -> sheet | nil [, err]

which est un nom ou un index entier 1-based. La feuille est parsée au premier accès puis mise en cache.

```
local first = assert(wb:sheet(1))
local data = assert(wb:sheet("Données"))
```

1.7.5 Valeurs et formules

1.7.5.1 sheet:read(row, col) read() conserve le contrat historique : il renvoie la valeur mise en cache de la cellule. Une formule sans cache renvoie donc nil.

```
local value = sheet:read(0, 0)
```

Valeurs possibles : chaîne, nombre, booléen, date ISO, date-heure ISO ou nil.

1.7.5.2 sheet:get_formula(row, col) -> formula | nil

```
local formula = sheet:get_formula(3, 1)
if formula then
    print(formula.expression)
    print(formula.cached_value)
    print(formula.formula_type)
end
```

Champs exposés :

- expression : expression sans = ;
- cached_value : valeur brute mise en cache ou nil ;
- formula_type : normal, shared, array, etc. ;
- ref : plage associée lorsqu'elle existe ;
- shared_index : identifiant d'une formule partagée lorsqu'il existe.

Une formule partagée secondaire peut ne pas contenir d'expression résolue. Elle est inspectable, mais la réécrire directement provoque une erreur plutôt que de produire une formule incorrecte.

Copier une formule ordinaire dans un nouveau classeur :

```
local formula = source:get_formula(3, 1)
if formula and formula.expression then
    destination:write(3, 1, formula)
end
```

1.7.6 Lecture des styles

1.7.6.1 sheet:get_style(row, col) -> style | nil

```

local style = sheet:get_style(0, 0)
if style then
    print(style.bold, style.font_name, style.font_size)
    print(style.number_format)
    if style.border and style.border.bottom then
        print(style.border.bottom.style)
    end
end

```

Le style retourné peut être passé directement à `write()` ou `set_style()`. Seul le sous-ensemble pris en charge est exposé : police simple, couleurs RGB, fond plein, alignement, format numérique et quatre bordures. Les thèmes, couleurs indexées, diagonales et autres variantes avancées peuvent être omis.

1.7.7 Lecture de la mise en page

1.7.7.1 `sheet:get_column_width(col)`

```
print(sheet:get_column_width(0))
```

Renvoie nil si aucune largeur explicite n'est enregistrée.

1.7.7.2 `sheet:get_row_height(row)`

```
print(sheet:get_row_height(0))
```

1.7.7.3 `sheet:get_frozen_panes() -> rows, cols`

```

local rows, cols = sheet:get_frozen_panes()
print(rows, cols)

```

1.7.7.4 `sheet:get_auto_filter() -> range | nil`

```
print(sheet:get_auto_filter())
```

1.7.8 Lecture des cellules fusionnées

1.7.8.1 `sheet:merged_cells() -> { range, ... }`

```

for _, range in ipairs(sheet:merged_cells()) do
    print(range)
end

```

Les plages sont normalisées en notation A1, par exemple A1:D1.

1.7.9 Lecture des hyperliens

1.7.9.1 `sheet:get_hyperlink(row, col) -> table | nil`

```

local link = sheet:get_hyperlink(2, 4)
if link then
    print(link.target, link.internal, link.tooltip, link.ref)
end

```

1.7.9.2 `sheet:hyperlinks() -> { table, ... }`

```

for _, link in ipairs(sheet:hyperlinks()) do
    print(link.ref, link.target)
end

```

1.7.10 Dimensions et itération

1.7.10.1 sheet:dims()

```
local maxrow, maxcol = sheet:dims()
```

Une feuille vide renvoie -1, -1.

1.7.10.2 sheet:rows()

```
for row in sheet:rows() do
    for col = 1, row.n do print(row[col]) end
end
```

L'itérateur renvoie les valeurs mises en cache et conserve les lignes vides intermédiaires.

1.8 Limites et validation ZIP

Options de `xlsx.open` :

Option	Défaut	Plafond accepté
max_file_size	256 Mio	pas de plafond interne supplémentaire
max_entries	10 000	
max_entry_size	64 Mio	8 Gio
max_total_size	512 Mio	64 Gio
max_path_length	4 096 octets	1 Mio
max_total_name_bytes	16 Mio	64 Mio
max_compression_ratio	200	1 000 000 000
use_babet	true	booléen
validate_archive	true	booléen

Un exemple par limite :

```
xlsx.open("a.xlsx", { max_file_size = 32 * 1024 * 1024 })
xlsx.open("a.xlsx", { max_entries = 2000 })
xlsx.open("a.xlsx", { max_entry_size = 16 * 1024 * 1024 })
xlsx.open("a.xlsx", { max_total_size = 128 * 1024 * 1024 })
xlsx.open("a.xlsx", { max_path_length = 1024 })
xlsx.open("a.xlsx", { max_total_name_bytes = 1024 * 1024 })
xlsx.open("a.xlsx", { max_compression_ratio = 100 })
```

Exemple combiné pour un fichier reçu d'un tiers :

```
local wb, err = xlsx.open("import.xlsx", {
    max_file_size = 64 * 1024 * 1024,
    max_entries = 5000,
    max_entry_size = 16 * 1024 * 1024,
    max_total_size = 128 * 1024 * 1024,
    max_path_length = 2048,
    max_total_name_bytes = 2 * 1024 * 1024,
    max_compression_ratio = 100,
    use_babet = true,
    validate_archive = true,
})
assert(wb, err)
```

Le lecteur Lua refuse notamment :

- ZIP multidisque, ZIP64, chiffrement et méthodes autres que STORED/DEFLATE ;
- noms absolus, traversées `..`, backslashes, doublons et noms non UTF-8 ;
- en-têtes locaux incohérents, data descriptors incohérents et chevauchements ;
- tailles annoncées incohérentes, CRC incorrects et flux DEFLATE tronqués ;

- sortie décompressée ou rapport de compression au-delà des limites.

1.9 DataFrame

Un DataFrame possède :

- columns : noms ordonnés, chaînes non vides et uniques ;
- rows : records nom -> valeur.

1.9.1 Construction

1.9.1.1 DF.from_rows(matrix [, opts]) Options :

Option	Effet
header	la première ligne fournit les noms
columns	noms explicites si header est faux

```
local d = DF.from_rows({
  { "nom", "âge" },
  { "Alice", 30 },
  { "Bob", 25 },
}, { header = true })
```

Les colonnes vides générées par un header nil deviennent colN. Les doublons de noms sont refusés.

1.9.1.2 DF.from_records(records [, columns])

```
local d = DF.from_records({
  { nom = "Alice", âge = 30 },
  { nom = "Bob", âge = 25 },
}, { "nom", "âge" })
```

Sans columns, les clés textuelles sont déduites et triées.

1.9.1.3 DF.from_sheet(sheet [, opts])

```
local d = DF.from_sheet(assert(wb:sheet("Données")), {
  header = true,
})
```

1.9.2 Introspection

```
print(d:nrow(), d:ncol())
print(table.concat(d:colnames(), ", "))
local ages = d:column("âge")
```

d:iter() renvoie une copie de chaque record :

```
for row in d:iter() do
  row.âge = 0 -- ne modifie pas d
end
```

1.9.3 Transformations non destructives

1.9.3.1 filter

```
local adults = d:filter(function(row)
  return row.âge >= 18
end)
```

Le callback reçoit une copie du record.

1.9.3.2 select

```
local names = d:select("nom")
local same = d:select({ "nom", "âge" })
```

Une même colonne ne peut pas être sélectionnée deux fois.

1.9.3.3 rename

```
local renamed = d:rename({ âge = "age" })
```

Une source inconnue, un nouveau nom vide ou une collision sont refusés.

1.9.3.4 mutate

```
local with_total = d:mutate("double_age", function(row)
  return row.âge * 2
end)
```

Le callback reçoit une copie du record source.

1.9.3.5 sort

```
local asc = d:sort("âge")
local desc = d:sort("âge", { desc = true })
```

Le tri est stable et place toujours les nil à la fin.

1.9.3.6 head et tail

```
local first = d:head(10)
local last = d:tail(10)
```

n doit être un entier positif ou nul. La valeur par défaut est 5.

1.9.4 Groupement et agrégations

```
local grouped = ventes:groupby("ville", "produit"):agg({
  total = { "sum", "montant" },
  moyenne = { "mean", "montant" },
  minimum = { "min", "montant" },
  maximum = { "max", "montant" },
  premier = { "first", "montant" },
  dernier = { "last", "montant" },
  nombre = { "count" },
})
```

Fonctions : sum, mean/avg, min, max, count, first, last.

Les clés de groupe prennent en charge nil, booléens, chaînes binaires et nombres. La sérialisation interne est préfixée par type et longueur ; une chaîne contenant des séparateurs ou des octets NUL ne peut donc plus fusionner deux groupes distincts.

groupby:count() équivaut à :

```
local counts = d:groupby("ville"):agg({ n = { "count" } })
```

1.9.5 Sorties

```
local records = d:to_records()
local rows = d:to_rows()
local rows_without_header = d:to_rows({ header = false })
print(d:tostring(20))
d:show(20)
```

1.10 Contrat d'erreur

Les erreurs d'utilisation lèvent une erreur Lua :

- mauvais type ou mauvaise option ;
- nom de colonne ou feuille invalide ;
- date impossible ;
- indice hors limites ;
- transformation DataFrame incohérente.

Les erreurs de système ou de données renvoient normalement nil, err :

```
local wb, err = xlsx.open("absent.xlsx")
if not wb then
    io.stderr:write(err, "\n")
end
```

workbook:sheet() suit le même principe pour une feuille corrompue ou une partie manquante.

Workbook:save() vérifie les résultats d'écriture, de flush, de fermeture et de publication. Il ne renvoie plus true après une écriture réellement échouée.

1.11 Tests et interopérabilité

./run_tests.sh

Le harnais vérifie notamment :

- écriture et relecture ;
- chaînes Unicode et entités XML ;
- CRC corrompu ;
- systèmes de dates 1900 et 1904 ;
- styles, formats numériques et dates stylées ;
- largeurs, hauteurs, volets figés, filtres et formules ;
- limites Excel et XML 1.0 ;
- appels automatiques à Babet ;
- non-destruction des DataFrames ;
- absence de collision dans groupby ;
- aller-retour réel avec openpyxl.

Babet est recherché dans cet ordre :

1. variable BABET_BIN ;
2. binaire local bin/babet ;
3. commande babet disponible dans le PATH.

Le script crée ensuite un venv Python temporaire, installe openpyxl>=3.1,<4, exécute l'interopérabilité séparément avec Babet et Lua standard lorsqu'ils sont disponibles, puis supprime le venv, le cache pip et les fichiers temporaires. Le nettoyage est assuré aussi lorsqu'un test échoue. Une installation globale d'openpyxl n'est donc ni utilisée ni nécessaire.

Prérequis de cette phase : python3, le module venv, pip dans le venv et un accès au dépôt Python configuré. Sous Debian ou Ubuntu, installer python3-venv si la création du venv échoue.

Utiliser le binaire local :

```
cp /chemin/vers/babet bin/babet
chmod +x bin/babet
./run_tests.sh
```

Forcer les runtimes ou la version d'openpyxl :

```
BABET_BIN=../babet/bin/babet ./run_tests.sh
LUA_BIN=/usr/bin/lua5.5 ./run_tests.sh
OPENPYXL_SPEC='openpyxl==3.1.5' ./run_tests.sh
```

1.12 Génération des PDF

```
cd doc  
./build_doc.sh
```

Sorties :

- documentation-fr.pdf ;
- documentation-en.pdf.

Prérequis : Pandoc et XeLaTeX ou LuaLaTeX. Le script utilise un répertoire temporaire et ne laisse pas les fichiers auxiliaires LaTeX dans le projet.

1.13 Limites connues

- Écriture ZIP STORED uniquement ; lecture STORED et DEFLATE.
- Pas de ZIP64 dans le parseur Lua interne.
- Pas de graphiques, images, validations de données, tableaux structurés ni mise en forme conditionnelle.
- Les formules sont lues et écrites mais ne sont pas calculées. Les formules partagées secondaires sans expression résolue ne sont pas réécrites.
- La lecture des styles couvre le sous-ensemble pris en charge ; les thèmes, couleurs indexées, diagonales et options avancées peuvent être omis.
- Pas de lecture en streaming : le fichier et les XML utiles restent en mémoire.
- Parsing XML spécialisé au sous-ensemble XLSX utilisé, pas parseur XML général.
- Pas de normalisation Unicode ou de case folding Unicode complet pour les noms de feuilles ; la détection insensible à la casse est fiable pour l'ASCII.